

Numerical Analysis Exercise 6

15. For the following linear system

$$\begin{cases} 4x_1 + 3x_2 = 24 \\ 3x_1 + 4x_2 - x_3 = 30 \\ -x_2 + 4x_3 = -24 \end{cases}$$

(1) write the Jacobi and Gauss-Seidel iterative scheme, respectively.

Solution:

Jacobi iterative scheme:

$$\begin{cases} x_1^{k+1} = 6 - \frac{3}{4}x_2^k \\ x_2^{k+1} = \frac{15}{2} - \frac{3}{4}x_1^k + \frac{1}{4}x_3^k \\ x_3^{k+1} = -6 + \frac{1}{4}x_2^k \end{cases}$$

Gauss-Seidel iterative scheme:

$$\begin{cases} x_1^{k+1} = 6 - \frac{3}{4}x_2^k \\ x_2^{k+1} = \frac{15}{2} - \frac{3}{4}x_1^{k+1} + \frac{1}{4}x_3^k \\ x_3^{k+1} = -6 + \frac{1}{4}x_2^{k+1} \end{cases}$$

(2) perform four iterations on each of the scheme given in part(1) and determine which scheme you think gives the better approximation to the solution

Solution:

```
1  clc;
2  clear all;
3  close all;
4
5  A = [4,3,0;3,4,-3;0,-1,4];
6  b = [24,30,-24];
```

```

7 % A = [3,-1,1;3,6,2;3,3,7];
8 % b = [1,0,4];
9 b = b';
10 fprintf("真实解为:")
11 x=(A\b)'
12 %A = input("Please input the A:")
13 %b = input("Please input the b:")
14
15 ab = [A,b]%a为系数矩阵, b为右端项
16 N = 100;%设置为迭代次数
17
18 split_J=Split_J(A) %用来计算|TJ| 判断雅可比迭代是否收敛
19 split_G=Split_G(A)%用来计算|TG| 判断高斯迭代是否收敛
20
21 if(split_J<1)
22     J=Jacobi_Iteration(ab,N)%Jacobi迭代法
23     error_J = E(J,X);
24     error_J = error_J(:,4);
25 else
26     fprintf("because of the split_J > 1,so we can't use Gauss
method")
27 end
28
29 if(split_G<1)
30     G = Gauss_Iteration(ab,N)%高斯赛德尔迭代法
31     error_G = E(G,X);
32     error_G = error_G(:,4);
33 else
34     fprintf("because of the split_G > 1,so we can't use Gauss
method")
35 end
36
37 function G = Gauss_Iteration(ab,N)
38     [R,C] = size(ab);
39     G = zeros(R);%设G为高斯赛德尔迭代法的解集
40     G(1,:) = [1,1,1];
41     G(2,:) = [-1,0,1];
42     for i = 1:R
43         ab(i,:) = ab(i,:)./ab(i,i);
44     end
45     for i = 1:R
46         ab(i,i) = 0;%斜角赋值为0

```

```

47     end
48
49     fprintf("形式处理后的增广矩阵:");
50     ab(1:R,1:R) = -ab(1:R,1:R);
51
52     for k = 2:N+1
53         for i = 1:R
54             G(k,i) = ab(i,1:R)*G(k-1,:)' + ab(i,C);
55             G(k-1,i) = G(k,i);
56         end
57     end
58     ab
59     fprintf("Gauss_Seidel的迭代解集:")
60     %G = G(5,:)
61 end
62
63 function J = Jacobi_Iteration(ab,N)
64     [R,C] = size(ab);
65     J = zeros(R);%设J为雅可比迭代法的解集
66
67     for i = 1:R
68         ab(i,:) = ab(i,:)./ab(i,i);
69     end
70     for i = 1:R
71         ab(i,i) = 0;%斜角赋值为0
72     end
73
74     fprintf("形式处理后的增广矩阵");
75     ab(1:R,1:R) = -ab(1:R,1:R);
76
77     for k = 1:N
78         J(k+1,:) = ab(1:R,1:R)*J(k,:)' + ab(1:R,C);
79     end
80
81     fprintf("Jacobi的迭代解集:")
82     %J = J(4,:)
83 end
84
85 function error = E(m,X) %用来计算迭代值与真实值误差
86     for i = 1:size(m,1)
87         for j = 1:size(m,2)
88             error(i,j) = (m(i,j)-X(1,j)).^2;

```

```

89         end
90         error(i,size(m,2)+1) = sqrt(sum(error(i,:)));
91     end
92 end
93
94 function split = Split_J(A) %用来计算|TJ|
95 [R,C] = size(A);
96 ab_D = A;
97 for i=1:R
98     for j=1:R
99         if(i~=j)
100             ab_D(i,j) = 0;
101         end
102     end
103 end
104
105 split = eig(ab_D^-1 * (ab_D-A))
106 split = max(split);
107
108 end
109
110 function split = Split_G(A) %用来计算|TG|
111 [R,C] = size(A);
112 ab_D = A;
113 ab_L = A;
114 for i=1:R
115     for j=1:R
116         if(i~=j)
117             ab_D(i,j) = 0;
118         end
119         if(i>j)
120             ab_L(i,j) = -ab_L(i,j);
121         else
122             ab_L(i,j) = 0;
123         end
124     end
125 end
126
127 ab_U = ab_D - ab_L - A;
128
129 split = eig((ab_D-ab_L)^-1 * ab_U)
130 split = max(split);

```

131

132 **end**

```
1  真实解为:
2  X = 1×3
3      10.5000    -6.0000    -7.5000
4
5  Jacobi的迭代解集:
6  J = 101×3
7      0          0          0
8      6.0000     7.5000    -6.0000
9      0.3750    -1.5000    -4.1250
10     7.1250     4.1250    -6.3750
11     2.9062    -2.6250    -4.9688
12     7.9688     1.5938    -6.6562
13     4.8047    -3.4688    -5.6016
14     8.6016    -0.3047    -6.8672
15     6.2285    -4.1016    -6.0762
16     9.0762    -1.7285    -7.0254
17
18 Gauss_Seidel的迭代解集:
19 G = 100×3
20     6.0000     3.0000    -5.2500
21     3.7500     0.7500    -5.8125
22     5.4375    -0.9375    -6.2344
23     6.7031    -2.2031    -6.5508
24     7.6523    -3.1523    -6.7881
25     8.3643    -3.8643    -6.9661
26     8.8982    -4.3982    -7.0995
27     9.2986    -4.7986    -7.1997
28     9.5990    -5.0990    -7.2747
29     9.8242    -5.3242    -7.3311
30
31 split = 3×1
32     -0.8660
33      0.8660
34     -0.0000
35
36 ans = 0.8660
37
```

we using the Euclidean distance to discribe approximation , absolutely to find that the Gauss_Seidel is the better method;

```
1 error_J = 101×1
2     14.2302
3     14.3091
4     11.5826
5     10.7318
6      8.6869
7      8.0489
8      6.5152
9      6.0366
10     4.8864
11     4.5275
12 error_G = 100×1
13     10.3108
14      9.6939
15      7.2705
16      5.4528
17      4.0896
18      3.0672
19      2.3004
20      1.7253
21      1.2940
22      0.9705
```

16. Test the convergence of the Jacobi and Gauss-Seidel iterative method for solving the following linear systems.

$$(1) \begin{cases} 2x_1 - x_2 + x_3 = -1 \\ x_1 + x_2 + x_3 = 2 \\ -x_1 - 2x_2 + 2x_3 = -5. \end{cases}$$

```
1 clc;
2 clear all;
3 close all;
4
5 %A = [4,3,0;3,4,-3;0,-1,4];
6 %b = [24,30,-24];
7 A = [3,-1,1;3,6,2;3,3,7];
```

```

8  b = [1,0,4];
9  b = b';
10 fprintf("真实解为:")
11 x=(A\b) '
12 %A = input("Please input the A:")
13 %b = input("Please input the b:")
14
15 ab = [A,b]%a为系数矩阵, b为右端项
16 N = 100;%设置为迭代次数
17
18 split_J=Split_J(A) %用来计算|TJ| 判断雅可比迭代是否收敛
19 split_G=Split_G(A)%用来计算|TG| 判断高斯迭代是否收敛
20
21 if(split_J<1)
22     J=Jacobi_Iteration(ab,N)%Jacobi迭代法
23     error_J = E(J,X);
24     error_J = error_J(:,4);
25 else
26     fprintf("because of the split_J > 1,so we can't use Gauss
method")
27 end
28
29 if(split_G<1)
30     G = Gauss_Iteration(ab,N)%高斯赛德尔迭代法
31     error_G = E(G,X);
32     error_G = error_G(:,4);
33 else
34     fprintf("because of the split_G > 1,so we can't use Gauss
method")
35 end

```

other codes is showed in last answer.

```

1  真实解为:
2  X = 1×3
3      0.0351    -0.2368    0.6579
4
5  Jacobi的迭代解集:
6  J = 101×3
7      0          0          0
8      0.3333      0      0.5714

```

```

 9      0.1429   -0.3571   0.4286
10      0.0714   -0.2143   0.6633
11      0.0408   -0.2568   0.6327
12      0.0368   -0.2313   0.6640
13      0.0349   -0.2398   0.6548
14      0.0352   -0.2357   0.6592
15      0.0350   -0.2373   0.6574
16      0.0351   -0.2366   0.6581
17
18 Gauss_Seidel的迭代解集:
19 G = 100x3
20      2.6667    0.3333   -0.7143
21      0.6825   -0.1032    0.3231
22      0.1912   -0.2033    0.5766
23      0.0734   -0.2289    0.6381
24      0.0443   -0.2349    0.6531
25      0.0374   -0.2364    0.6567
26      0.0356   -0.2367    0.6576
27      0.0352   -0.2368    0.6578
28      0.0351   -0.2368    0.6579
29      0.0351   -0.2368    0.6579
30
31
32 split = 3x1 complex
33      -0.4193 + 0.0000i
34      0.2097 + 0.1132i
35      0.2097 - 0.1132i
36
37 ans = -0.4193

```

the $\|T_J\| = D^{-1}(L + U) = 0.4193 < 1$, so it is easy to see that the solution is convergent.

$$(2) \begin{cases} x_1 + x_2 - 2x_3 = 1 \\ x_1 + x_2 + x_3 = 1 \\ 2x_1 + 2x_2x_3 = 1. \end{cases}$$

By using the the above codes.

```

1  真实解为:
2  x = 1x3
3      -3      3      1

```



```

4
5 split_J = -5.4047e-06 + 9.3612e-06i
6 split = 3×1
7     0
8     2
9     2
10
11 split_G = 2
12 Jacobi的迭代解集:
13 J = 101×3
14     0     0     0
15     1     1     1
16     1    -1    -3
17    -3     3     1
18    -3     3     1
19    -3     3     1
20    -3     3     1
21    -3     3     1
22    -3     3     1
23    -3     3     1
24
25 because of the split_G > 1,so we can't use Gauss method

```

through Function split_G ,we can know the $|T_G| = 2 > 1$, so we can't use the Gauss_iteration method.

```

1 split_G = 3×1
2     0
3     2
4     2

```

17. Determine the numbers of iterations necessary of the Jacobi and Gauss-Seidel iterative method with accuracy 10^{-4} in the l_∞ norm of solving the system.

$$\begin{cases} 3x_1 - x_2 + 2x_3 = 1 \\ 3x_1 + 6x_2 + 2x_3 = 0 \\ 3x_1 + 3x_2 + 7x_3 = 4. \end{cases}$$

with initial approximation vector $X^{(0)} = (0, 0, 0)^T$

```
1  clc;
2  clear all;
3  close all;
4
5  A = [3,-1,1;3,6,2;3,3,7];
6  b = [1,0,4];
7  b = b';
8  fprintf("真实解为:")
9  x=(A\b) '
10 %A = input("Please input the A:")
11 %b = input("Please input the b:")
12
13 ab = [A,b];%a为系数矩阵, b为右端项
14 N = 100;%设置为迭代次数
15 Epsilon =1e-4;
16 [R,C] = size(A);
17 ab_D = A;
18 ab_L = A;
19 for i=1:R
20     for j=1:R
21         if(i~=j)
22             ab_D(i,j) = 0;
23         end
24         if(i>j)
25             ab_L(i,j) = -ab_L(i,j);
26         else
27             ab_L(i,j) = 0;
28         end
29     end
30 end
31
32 A
33 ab_U = ab_D - ab_L - A
34 ab_D
35 ab_L
36
37 T_J = ab_D^-1*(ab_L + ab_U)
38 C_J = ab_D^-1*b;
39
40 T_G = (ab_D - ab_L)^-1 * ab_U
```

```

41 C_G = (ab_D-ab_L)^-1*b;
42
43 X_G = [0;0;0];
44 X_J = [0;0;0];
45 error_G = 1;
46 error_J = 1;
47
48 if(Split_G(A)<1)
49     i = 1;
50     while(error_G>=Epsilon)
51         X_G(:,i+1) = T_G*X_G(:,i) + C_G;
52         error_G(i) = sqrt(sum((X_G(:,i+1)-X(1,:))'.^2,1));
53         i = i+1;
54     end
55     X_G = X_G'
56 else
57     fprintf("Jacobi method can't convergent")
58 end
59
60 if(Split_J(A)<1)
61     i = 1;
62     while(error_J>=Epsilon)
63         X_J(:,i+1) = T_J*X_J(:,i) + C_J;
64         error_J(i) = sqrt(sum((X_J(:,i+1)-X(1,:))'.^2,1));
65         i = i+1;
66     end
67     X_J = X_J'
68 else
69     fprintf("Gauss method can't convergent")
70 end
71
72
73
74 function split = Split_J(A) %用来计算|TJ|
75 [R,C] = size(A);
76 ab_D = A;
77 for i=1:R
78     for j=1:R
79         if(i~=j)
80             ab_D(i,j) = 0;
81         end
82     end

```

```

83 end
84
85 split = eig(ab_D^-1 * (ab_D-A));
86 split = max(split);
87
88 end
89
90 function split = Split_G(A) %用来计算|TG|
91 [R,C] = size(A);
92 ab_D = A;
93 ab_L = A;
94 for i=1:R
95     for j=1:R
96         if(i~=j)
97             ab_D(i,j) = 0;
98         end
99         if(i>j)
100             ab_L(i,j) = -ab_L(i,j);
101         else
102             ab_L(i,j) = 0;
103         end
104     end
105 end
106
107 ab_U = ab_D - ab_L - A;
108
109
110
111 split = eig((ab_D-ab_L)^-1 * ab_U);
112 split = max(split);
113
114 end
115

```

```

1  真实解为:
2  X = 1×3
3      0.0351    -0.2368    0.6579
4
5
6  error_G = 1×7

```

7	0.3447	0.0866	0.0205	0.0051	0.0012	0.0003
	0.0001					
8						
9	X_G = 8×3					
10	0	0	0			
11	0.3333	-0.1667	0.5000			
12	0.1111	-0.2222	0.6190			
13	0.0529	-0.2328	0.6485			
14	0.0396	-0.2360	0.6556			
15	0.0361	-0.2366	0.6573			
16	0.0354	-0.2368	0.6578			
17	0.0352	-0.2368	0.6579			
18						
19	error_J = 1×11					
20	0.3905	0.2805	0.0431	0.0327	0.0084	0.0043
	0.0017	0.0007	0.0003	0.0001	0.0001	
21						
22	X_J = 12×3					
23	0	0	0			
24	0.3333	0	0.5714			
25	0.1429	-0.3571	0.4286			
26	0.0714	-0.2143	0.6633			
27	0.0408	-0.2568	0.6327			
28	0.0368	-0.2313	0.6640			
29	0.0349	-0.2398	0.6548			
30	0.0352	-0.2357	0.6592			
31	0.0350	-0.2373	0.6574			
32	0.0351	-0.2366	0.6581			
33						